

Cryptography I



DTU Compute

Department of Applied Mathematics and Computer Science

$$\Delta \int_a^b \varepsilon \Theta^{\sqrt{17}} + \Omega \int \delta e^{i\pi} = \{2.7182818284\}$$

$$f^{(i)}(x) = \infty$$

$$\chi^2$$

$$\Sigma!$$

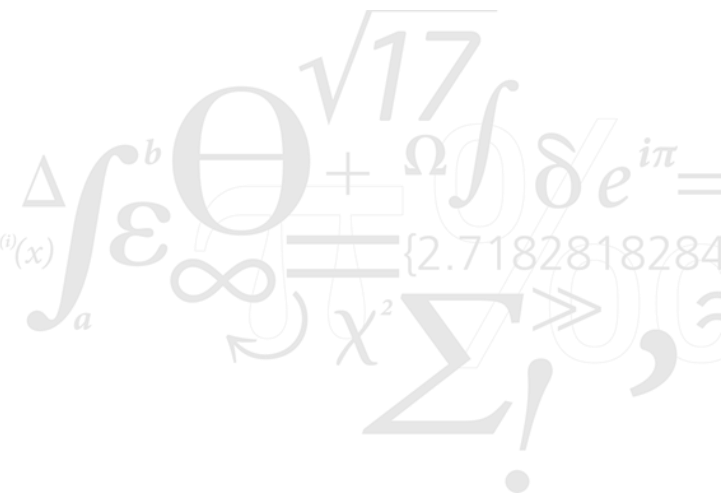
Course Overview

- **Lectures:** Room: LY.306.034.AUD and LY.306.035.AUD
- **Exercises:** Allocated Rooms:
 - **LY.303A.47.CR (Only allocated for 02339),**
 - LY.308.001.CR,
 - LY.308.009.CR,
 - LY.308.017.CR,
 - LY.308.101.EXP,
 - LY.308.109.EXP,
 - LY.308.117.EXP,
 - LY.308.127.CR

Examination

- Final exam (multiple choice)

- No AID- No notes, no books etc
- 2 Hours Exam



What Cryptography can do

- Cryptography is just one of the tools in the security tool box
 - But a very versatile and powerful tool
 - Cryptography can help solve important problems:
 - Keeping secrets
 - *Messages on the network*
 - *Data stored on disks, memory cards, USB sticks, ...*
 - Ensuring integrity
 - *Messages on the network (Message Authentication Codes)*
 - *Data stored on disk, ...*
 - Authentication
 - *User authentication*
 - Protecting shared secrets during communication
 - *Message authentication*
 - Sender authentication
 - Message authentication
- combine sender authentication and message integrity

Keeping Secrets

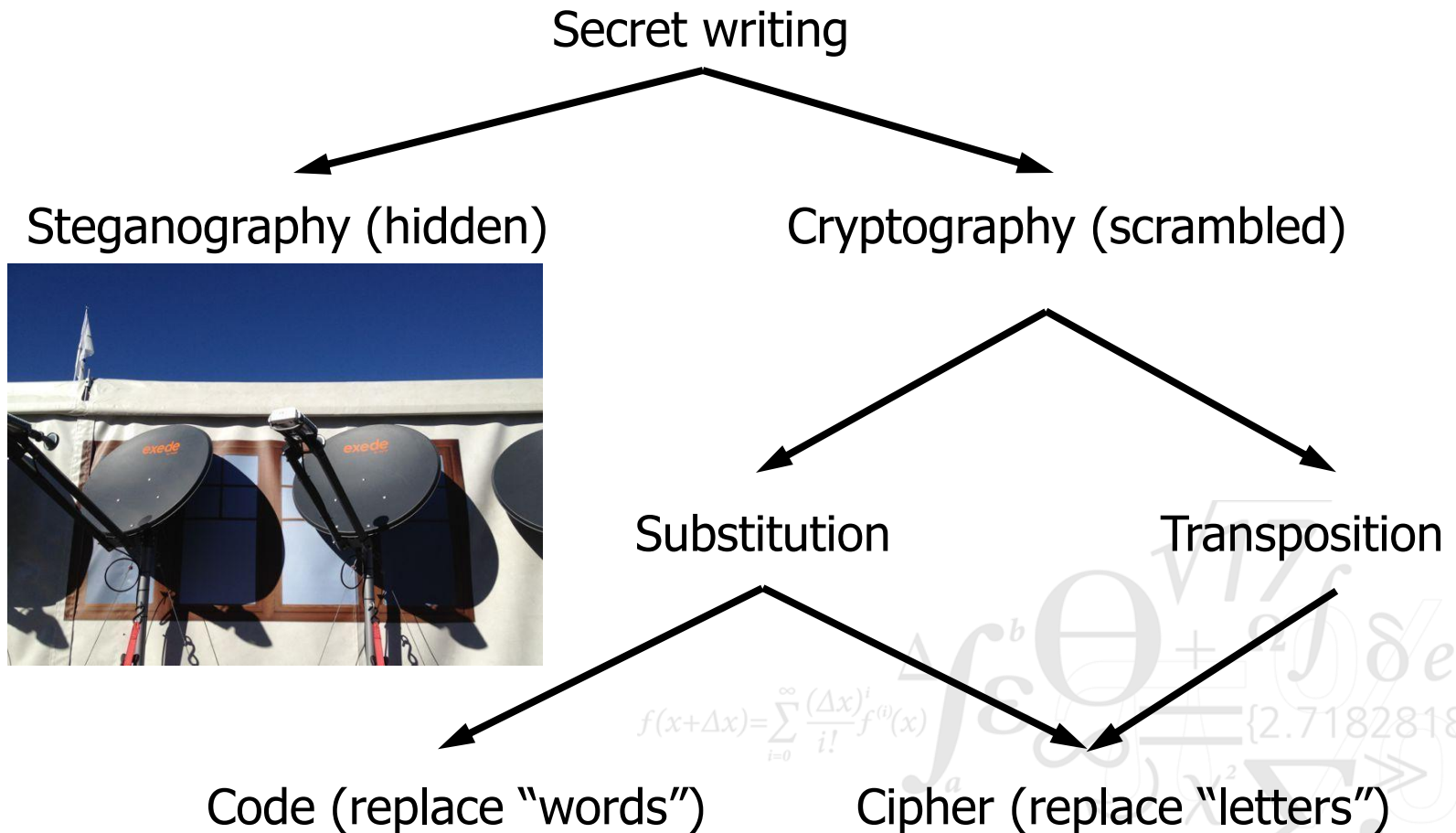
Steganography, Codes and Ciphers

- Steganography
 - Hide the existence of a secret message
 - Inconspicuous message (invisible ink, micro dots)
 - If the message is found, then the secret is revealed
- Codes
 - Replace symbols in the message with “codes”
 - Transformation must be agreed in advance
 - Inconspicuous if code is well defined
- Ciphers
 - Hide the meaning of the secret message
 - Scrambling according to an agreed algorithm
 - Conspicuous message (obviously a secret)



Cryptography

Fundamental principles



Source: <https://arstechnica.com/information-technology/2012/05/steganography-how-al-qaeda-hid-secret-documents-in-a-porn-video/>

Image Steganography Attack – Workflow

Based on: Menlo Security – “Image Steganography Example: How I Created an Attack”

Key Takeaway:

An image can look normal, but still carry hidden malicious data.

1 Prepare the payload

Attacker creates a malicious payload (e.g., reverse shell script or executable).



malicious.sh
(payload)

2 Hide the payload in an image

Use a steganography tool (e.g., steghide) to embed the payload inside an innocent-looking image (e.g., a photo).



```
steghide embed
-c image.jpg
-f malicious.sh
```

3 Distribute the image

The attacker shares the image through email, a website, or other channels. The image appears normal to users.



image.jpg
(looks normal)

4 Victim downloads the image

The victim receives and downloads the image, which looks like a legitimate photo. Most security tools do not detect the hidden payload.



5 Extract the payload

The attacker (or malware on the victim's system) uses a steganography tool to extract the hidden payload from the image.

```
steghide extract
-f image.jpg
```



malicious.sh
(recovered payload)

6 Execute the payload

The extracted payload is executed, giving the attacker unauthorized access (e.g., a reverse shell).



Attacker gains control of the system.

Why it works

- Steganography hides data within normal image pixels with minimal visible changes.
- The image looks legitimate to users and many security tools.
- Malicious data can be extracted later using the correct tool and password (if used).

Key tools (from the blog example)

- steghide** – to embed and extract hidden data
- A normal image file (e.g., JPG) as the carrier
- A malicious script or executable as the payload

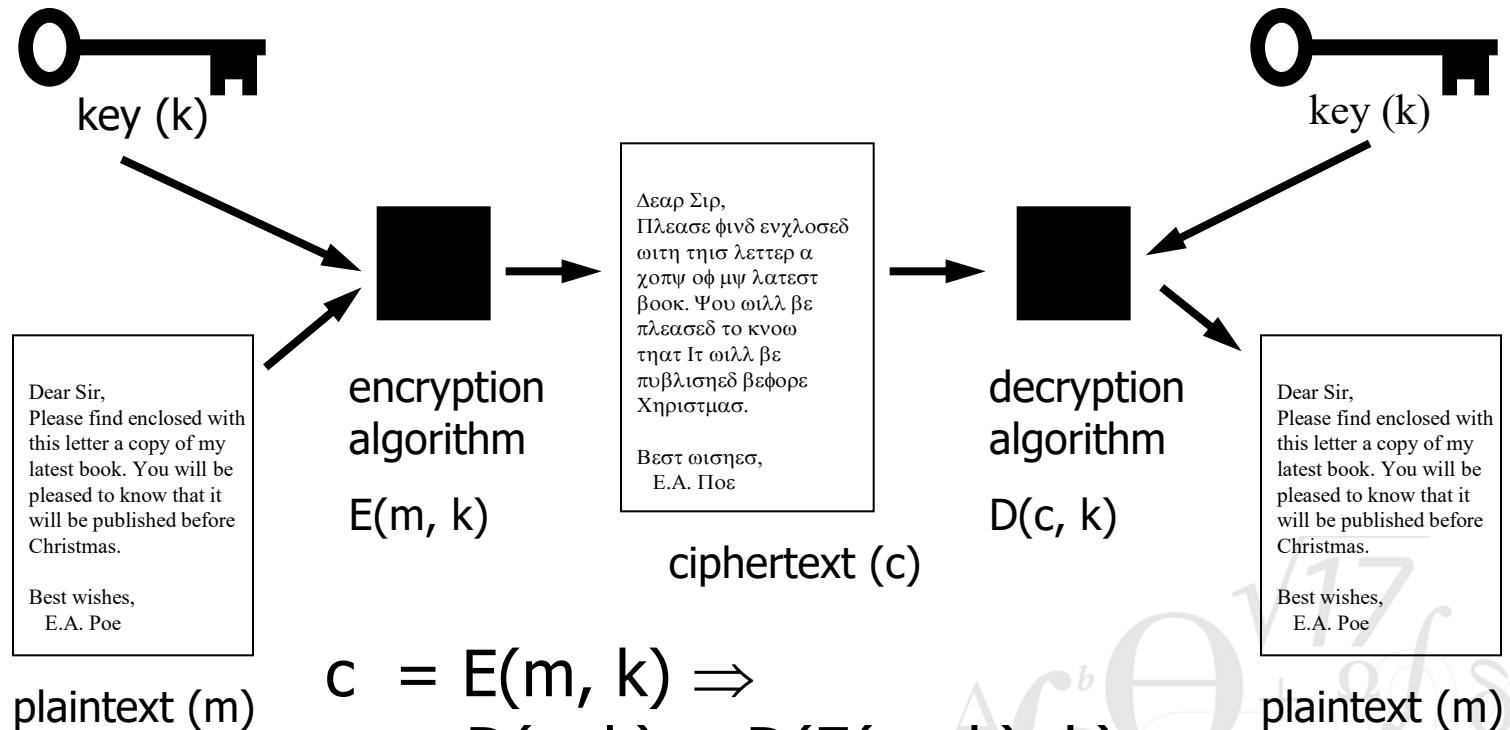
Defences

- Use advanced threat detection (e.g., Menlo Security Isolation) to analyse content in isolation.
- Inspect files for hidden data (steganalysis).
- Be cautious with unexpected images from untrusted sources.
- Use multi-layered security controls.

Same image. Different story.
Steganography can hide threats in plain sight.

Ref: Image is created with ChatGPT

Cipher = Algorithm + Key



$$c = E(m, k) \Rightarrow$$

$$m = D(c, k) = D(E(m, k), k)$$

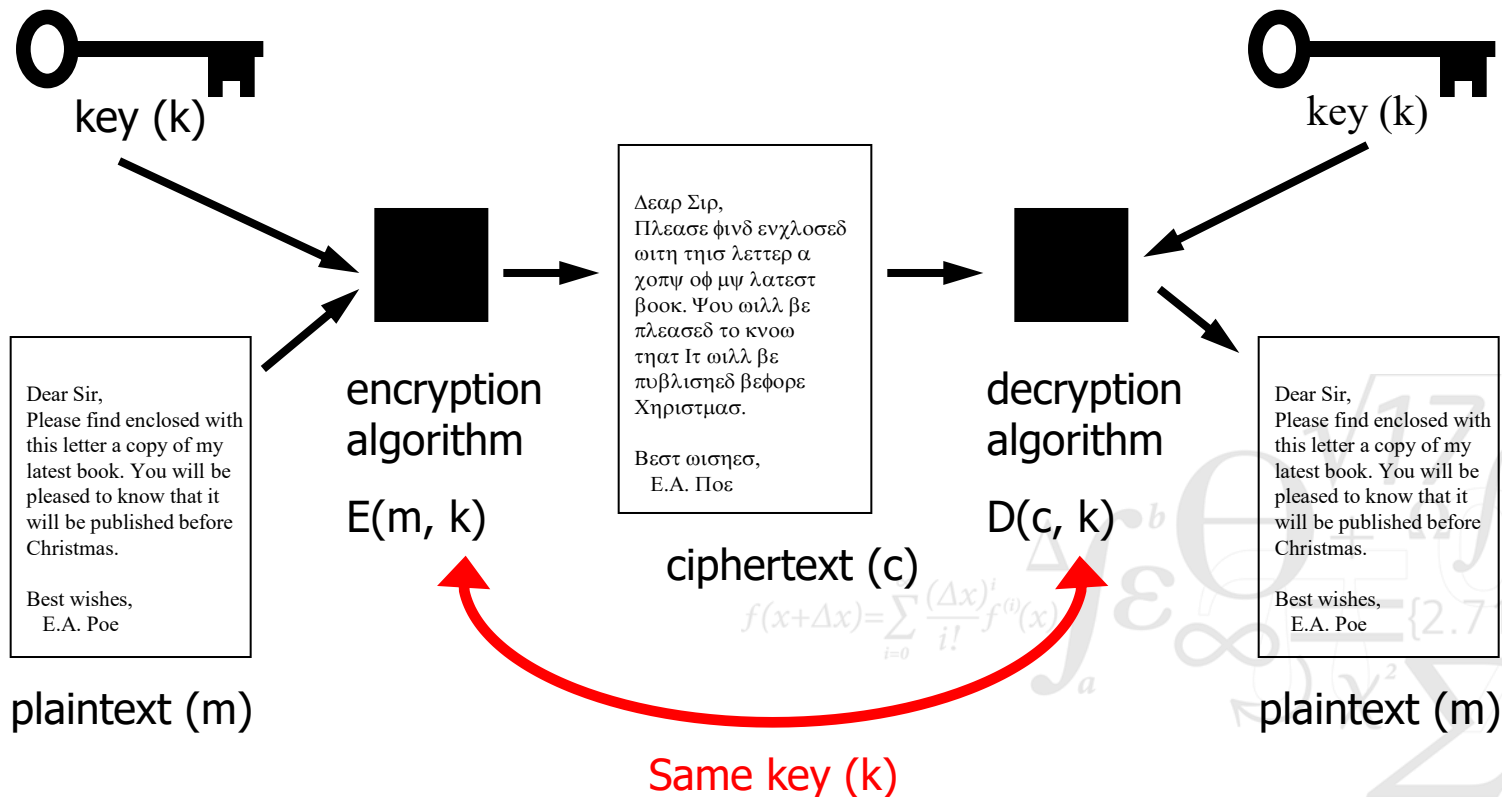
No cipher should rely on the secrecy of the algorithm!
A. Kerckoffs, "La Cryptographie Militaire", 1883

Basic Building Blocks of Cryptography

- Symmetric ciphers (1 key is used)
 - Same key is used for encryption and decryption
- Asymmetric ciphers (2 keys are used)
 - One key is used for encryption
 - Another key is used for decryption
 - *It is not computationally feasible to derive one key from the other*
- Cryptographic Hash Functions (no keys are used)
 - Scrambles input in a unique way
 - *Generates fixed length output from variable length input*
 - *Scrambles input ("decryption" is infeasible)*
- Digital signatures
 - One key signs data (private key)
 - Another key is used to verify signature (public key)
- Advanced algorithms, protocols and constructs not covered here

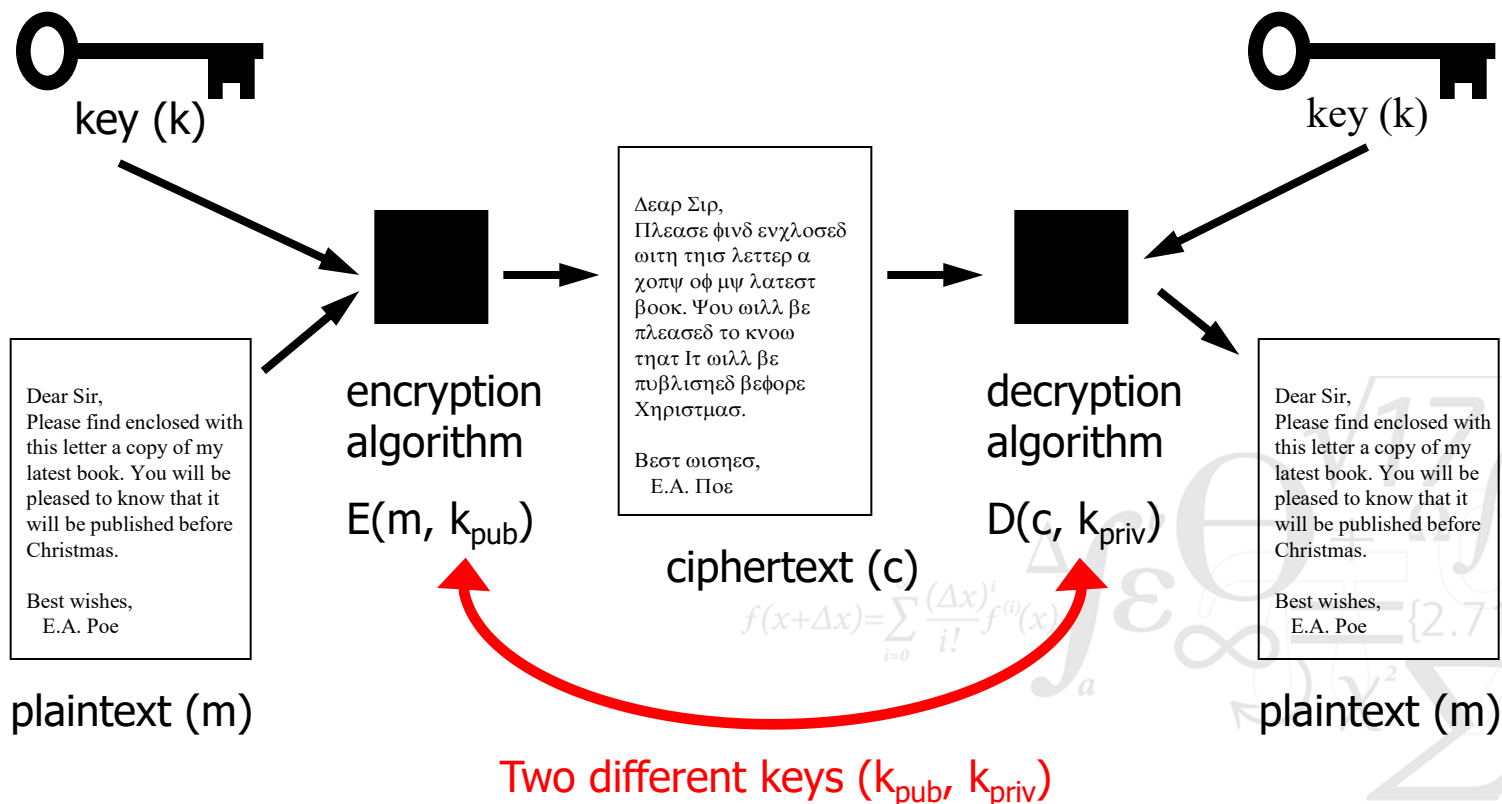
Symmetric Cryptography

- Decryption-key is identical to Encryption-key (or easily derived)

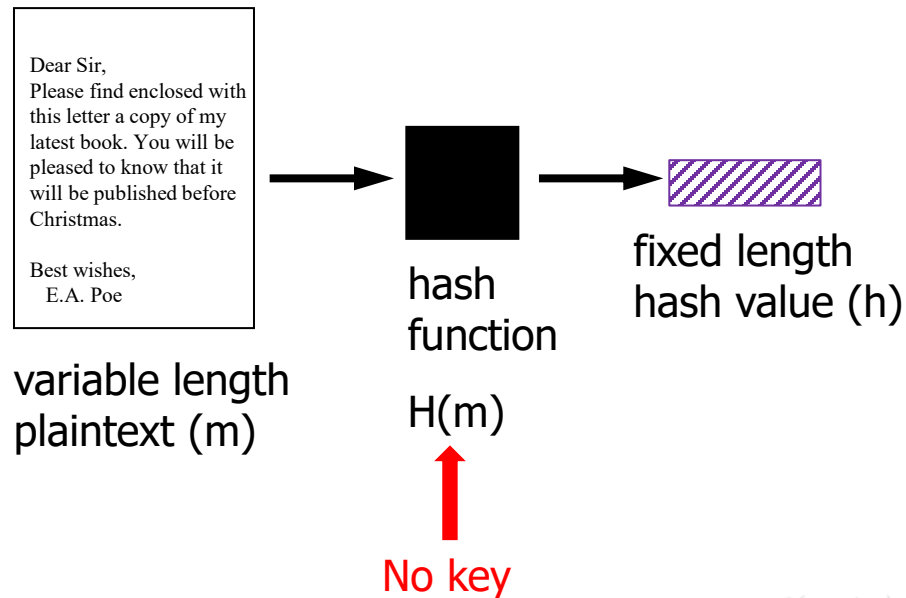


Asymmetric Cryptography

- Decryption-key cannot be derived from Encryption-key



Cryptographic Hash Functions



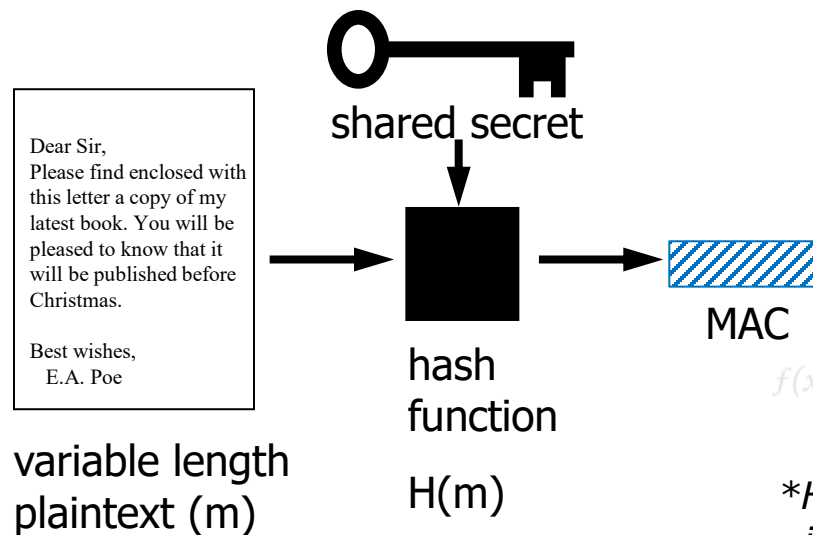
- Cryptographic hash functions must also satisfy:

1. Given M , computation of h is easy
2. Given h , it is intractable to compute M such that $H(M) = h$
3. It is intractable to find M and M' such that $H(M') = H(M)$

- Hash value h is often called a "fingerprint" or "digest" of M

Message Authentication Codes (MAC)

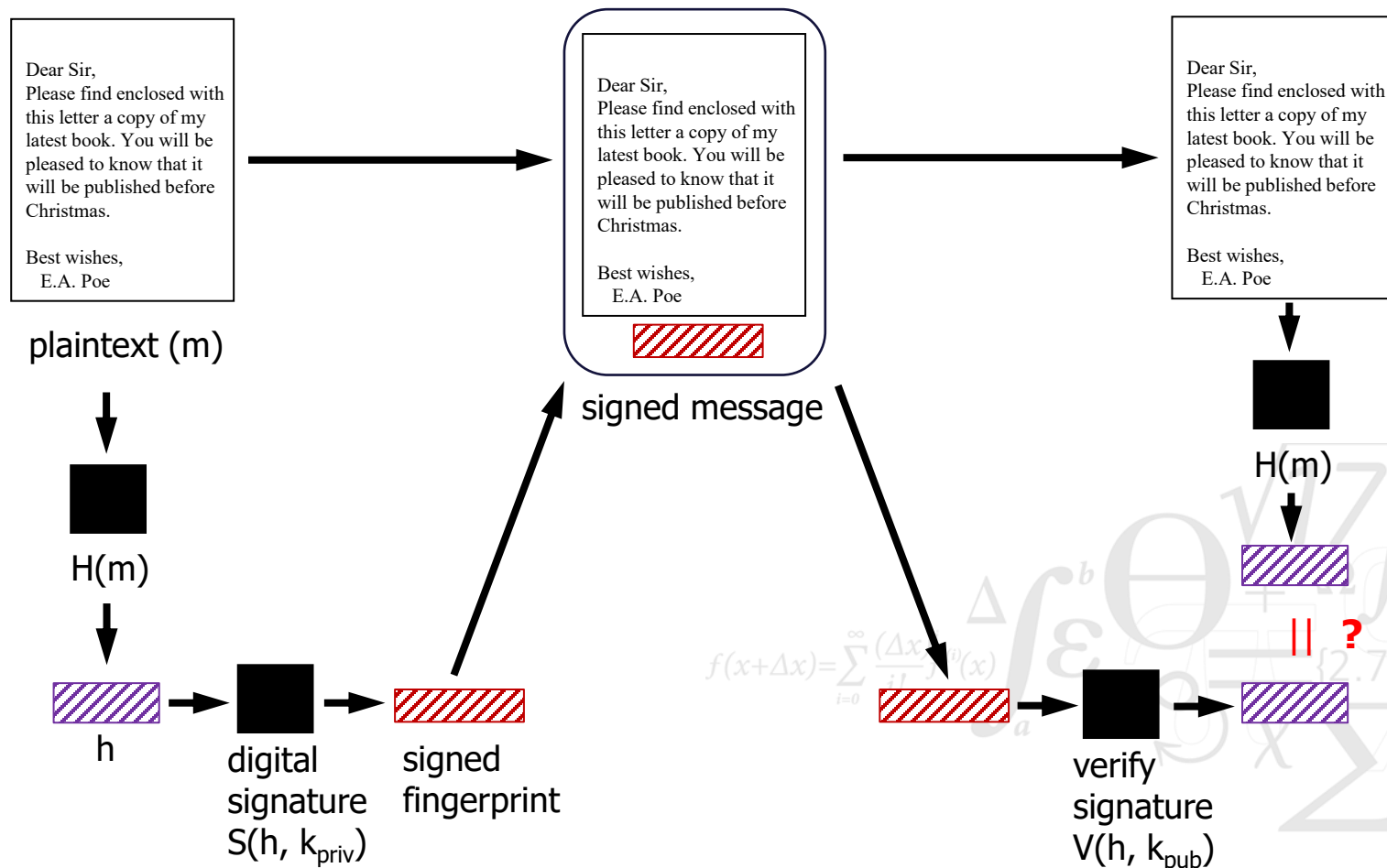
- Message Authentication Codes protect integrity of messages
- Hash-based MAC uses a cryptographic hash function and a shared secret
 - Shared secret (K) is prepended to the message (M) before applying H
 - $\text{MAC}(K, M) = H(K \parallel M \parallel K)^*$



**HMAC is a frequently used MAC function; it is defined in RFC 2104*

Digital Signatures

- Operation is expensive, so we normally sign the fingerprint



Security Properties

- Symmetric cryptography
 - Confidentiality of messages
 - Both parties know the shared key (may be more than two parties)
- Asymmetric cryptography
 - Confidentiality of messages
 - Only one party knows the secret key (only party who can decrypt M)
- Hash functions
 - Hash value corresponds to given M (with very high probability)
- Message Authentication Codes
 - Integrity of message (hash function)
 - Authenticity of message (shared secret)
- Digital signatures
 - Integrity and Authenticity (as with MAC)
 - Non-repudiation of message (assuming security of private-key)

Protection Goals

- Before considering a cryptographic solution, get clear about its protection goals:
 - Confidentiality
 - Integrity
 - Authenticity
 - Non-Repudiation
- People often say: “Our system provides secure communication” or “we need a secure communication channel”
What does this really mean?
- Cryptography is expensive, so we should only use the building blocks that we need

Characteristics of Good Ciphers

- Defined by Claude Shannon (1949)
 - The father of Information Theory



1. The amount of secrecy needed should determine the amount of labour appropriate for the encryption and decryption
2. The set of keys and the enciphering algorithm should be free from complexity
3. The implementation of the process should be as simple as possible
4. Errors in ciphering should not propagate and cause corruption of further information in the message
5. The size of the enciphered text should be no larger than the text of the original message

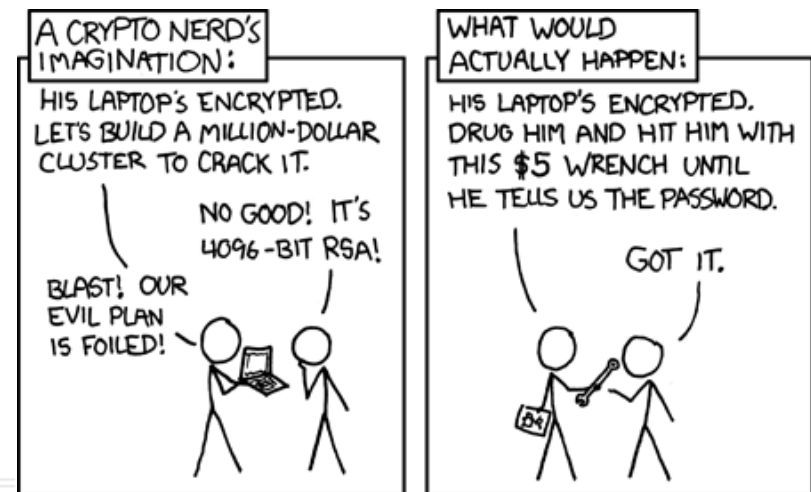
Security of Cryptographic Solutions

- There are 3 classes of attacks on crypto-systems
 - Attacking the cipher (algorithm and mode)
 - Attacking the key (key space, key generation, key management)
 - Attacking the cryptographic protocol
- Most crypto-systems are “computationally secure”
 - Security often measured in number of operations required by the best known attack (this is known as the “workload” of an attack)
 - *Strong algorithm, brute force attack $\Rightarrow$ workload = key-length/2*
 - Hardness of many algorithms is easily overcome with quantum computers

$$f(x+\Delta x) = \sum_{i=0}^{\infty} \frac{(\Delta x)^i}{i!} f^{(i)}(x)$$
$$\int_a^b \epsilon \Theta = \{2.7182818284\}$$
$$\chi^2$$
$$\sum$$
$$\gg$$
$$e^{i\pi}$$

Cryptanalysis attacking ciphers

- Cryptanalysis attempts to recover plaintext without access to key, but with full knowledge of algorithm
- There are four general types of cryptanalytic attacks:
 - Ciphertext-only attack
 - Known-plaintext attack
 - Chosen-plaintext attack
 - Adaptive chosen-plaintext attack
- Other attacks are equally effective
 - Rubber-hose cryptanalysis aka. purchase-key attack



<https://xkcd.com/538>

Cryptanalysis

attacking keys

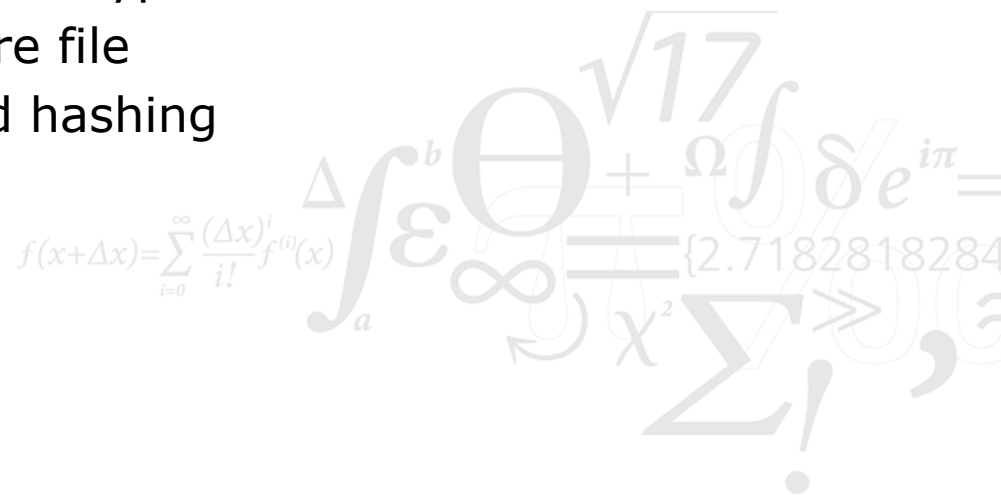
- Key space attacks
 - Exhaustive key search (brute force) attacks are possible with short keys
 - *DES has fixed 56 bit keys, which are cracked very quickly*
 - *AES uses 128, 192 or 256 bit keys (algorithm allows longer keys)*
 - Asymmetric algorithm keys are an order of magnitude longer
 - *Elliptic Curve crypto. achieves equivalent security with shorter keys*
- Key generation attacks
 - 128 bits = 16 random bytes are hard to remember
 - Key derivation functions (KDF) take a password and generates a key
 - *Guessing the password $\Rightarrow$ knowing the key*
- Key management
 - Keys must be stored, shared, distributed, ...
 - *All these steps may be attacked*

Q: A company wants to securely transfer a large file between two remote offices. It needs:

- secure establishment of a session key,
- efficient encryption of the large file,
- protection against modification of the encrypted data.

Which architecture is most appropriate?

- A. Use only hashing
- B. Use asymmetric cryptography for key establishment and authenticated symmetric encryption for the data
- C. Use only RSA for the entire file
- D. Use digital signatures and hashing



Q: A company has 20 branches. Each branch needs to communicate securely with every other branch using symmetric encryption.

Statement I: Pairwise symmetric keys can create a key-management scalability challenge as the number of communicating parties increases.

Statement II: Symmetric cryptography completely eliminates the need for key management

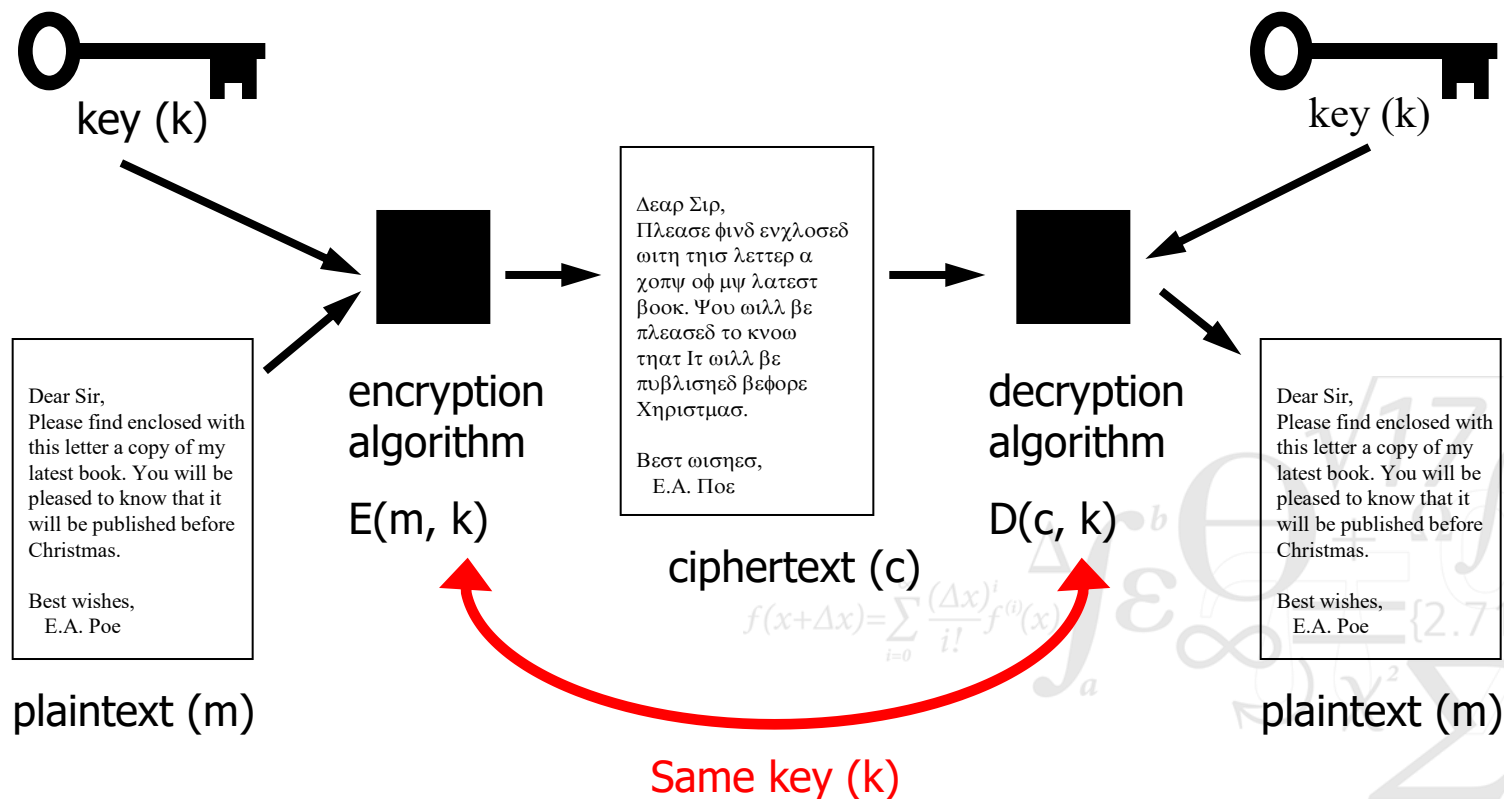
- A. Statement I is True, Statement II is True
- B. Statement I is True, Statement II is False
- C. Statement I is False, Statement II is True
- D. Statement I is False, Statement II is False

Coffee Break



Symmetric Cryptography

- Decryption-key is identical to Encryption-key (or easily derived)



One Time Pads

- A One Time Pad consists of a large non-repeating sequence of *truly random* characters
- Encryption xor each letter in the plaintext with the corresponding letter from the one time pad

$$C = P \oplus K$$

- Decryption xor each letter in the ciphertext with the corresponding letter from the one time pad

$$P = C \oplus K$$

- One time pads produce perfectly secure encryption
 - Known as information-theoretic security, cryptosystem cannot be broken by attacker with unlimited resources
 - *Derived from the work by Claude Shannon*

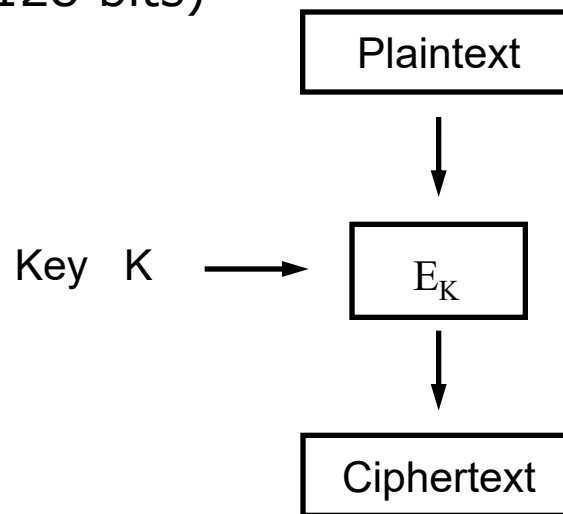


Symmetric Cryptography

- Two main classes of symmetric cryptography:
 - Block Ciphers
 - Streams Ciphers
- Well known Block Ciphers include:
 - DES (badly broken, but found in older textbooks)
 - *Keys are too short to protect against brute force*
 - Triple DES (3DES, still in use, but will be retired by 2023)
 - $C = E_{K3}(D_{K2}(E_{K1}(P)))$
 - AES (current standard adopted by NIST)
- Well known Stream Ciphers include:
 - A5/1 (used in GSM networks, essentially broken)
 - RC4 (difficult to use securely)
 - Block ciphers can be used to construct stream ciphers
 - *This is often the better option*

Block Cipher Algorithms

- Block ciphers operate on blocks of plaintext and ciphertext (usually 32, 64 or 128 bits)



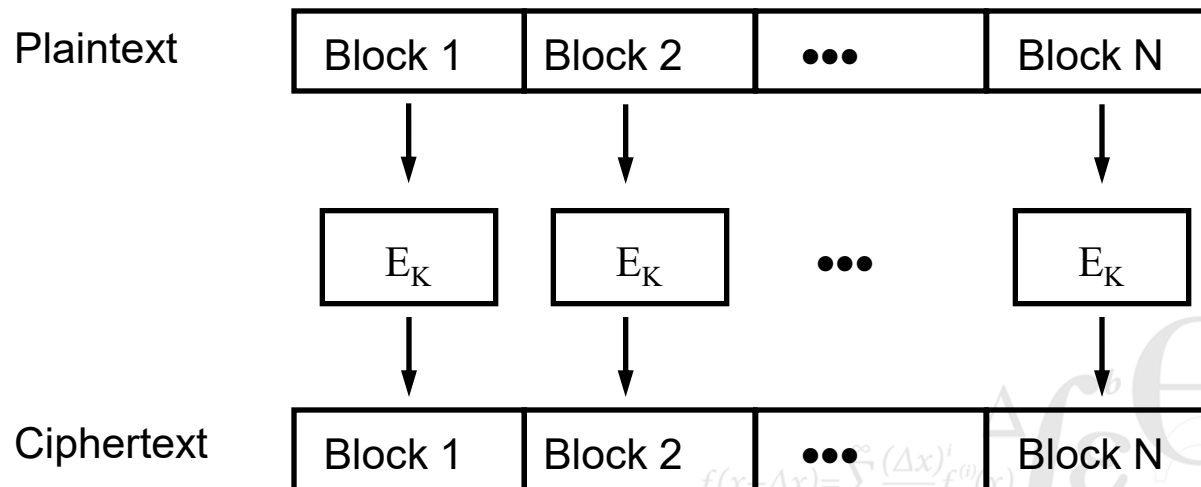
- Algorithm describes how to transform one block of a certain size
 - What happens when plaintext is shorter than block size of algorithm?
 - What happens when plaintext is longer than block size of algorithm?
- Introduces encryption schemes/modes

Algorithms and Schemes/Modes

- Cryptographic algorithms
 - Describe transformation of plaintext/ciphertext to ciphertext/plaintext
 - Do not describe how cryptography is used in systems
- Algorithms operate on fixed sized data
 - Blocks, stream units (bits/bytes)
- What happens when message size does not fit algorithm size?
 - Message is padded to fit algorithm size
- Cryptographic schemes/modes define:
 - How to encrypt plaintexts of arbitrary lengths
 - How to use initialisation vectors to make each encryption unique

Encrypting Long Plaintexts

- Divide the plaintext into blocks of the defined block size
 - Add padding to the end if necessary



Electronic Codebook Mode

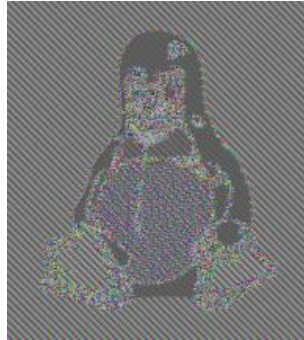
- ECB is the simplest application of a symmetric block cipher
 - plaintext blocks are separately encrypted into ciphertext blocks
 - *Figure on the previous slide*
 - the same plaintext block is always encrypted into the same ciphertext block (with the same key)
 - *Substitution of one "letter" for another "letter" => cipher*
 - Size of AES alphabet is 2^{128}
- Properties of ECB encryption
 - blocks can be en-/decrypted in random order
 - *ECB used in encrypted file systems means that the "seek" operation works*
 - Does require block alignment, but disk blocks are typically multiples of 32, 64 or 128 bits (e.g., 512B, 1024B or 4096B)
 - blocks can be en-/decrypted in parallel
 - *Useful in high speed network communication*

Problems with ECB

- Structure in the plaintext is reflected in the ciphertext



Tux



ECB
encoding



Non-ECB
encoding

- Messages often have stereotyped beginnings (Dear Foo) and endings (Best regards, Bar)
- Cryptanalyst who learn the encryption of a plaintext block can decrypt the corresponding ciphertext block in all messages encrypted with the same key
- ECB is also vulnerable to *block replay attacks*

Cipher Block Chaining Mode

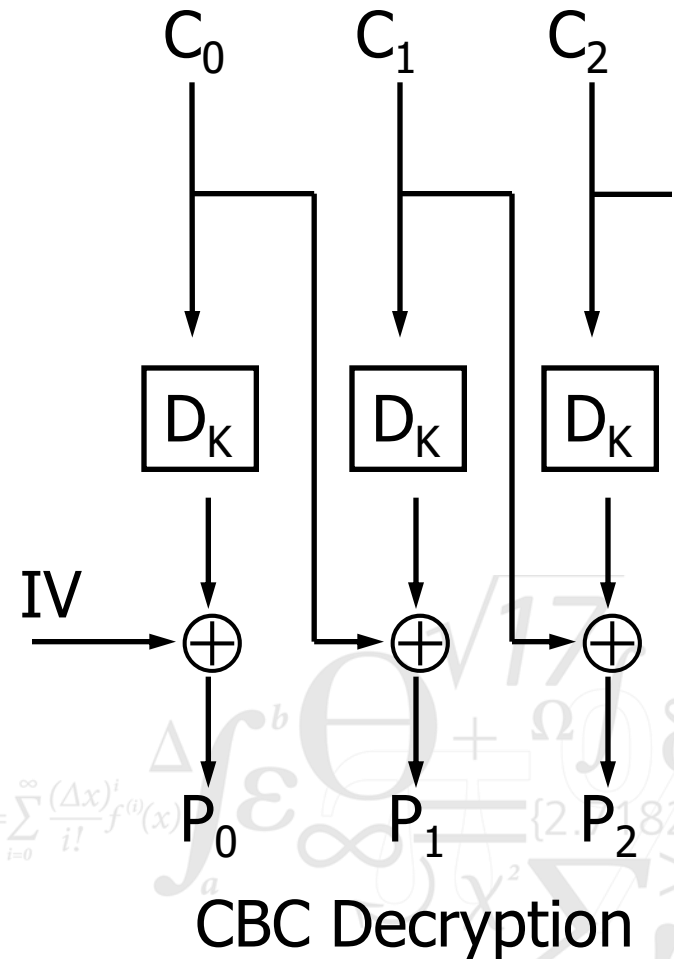
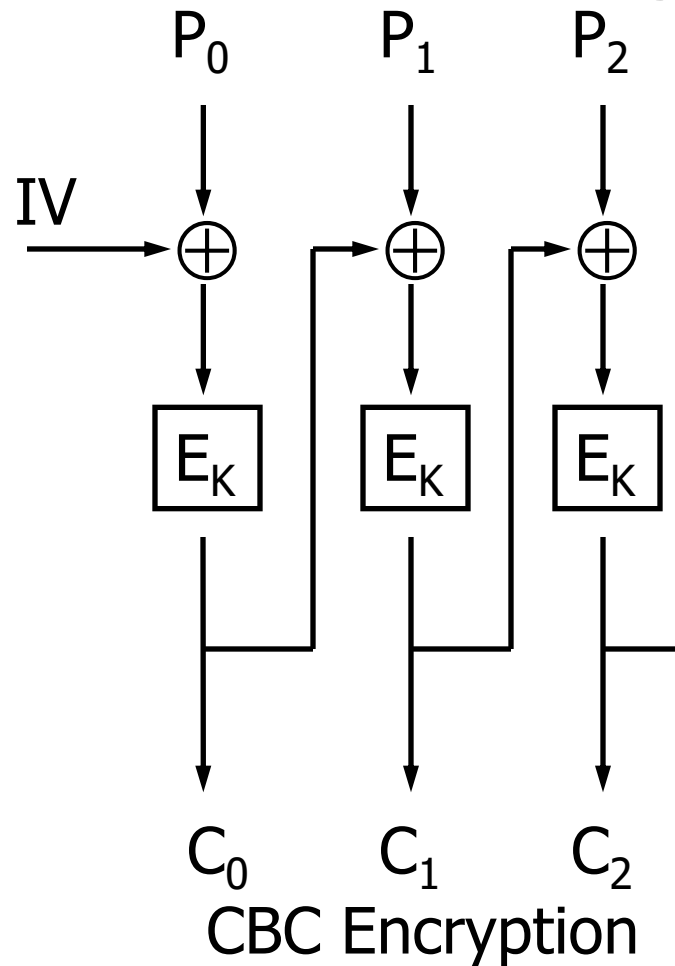
- Introduce a feedback mechanism
 - include the previous ciphertext block in the encryption of the current plaintext block

$$C_i = E_k(P_i \oplus C_{i-1})$$

$$P_i = C_{i-1} \oplus D_k(C_i)$$

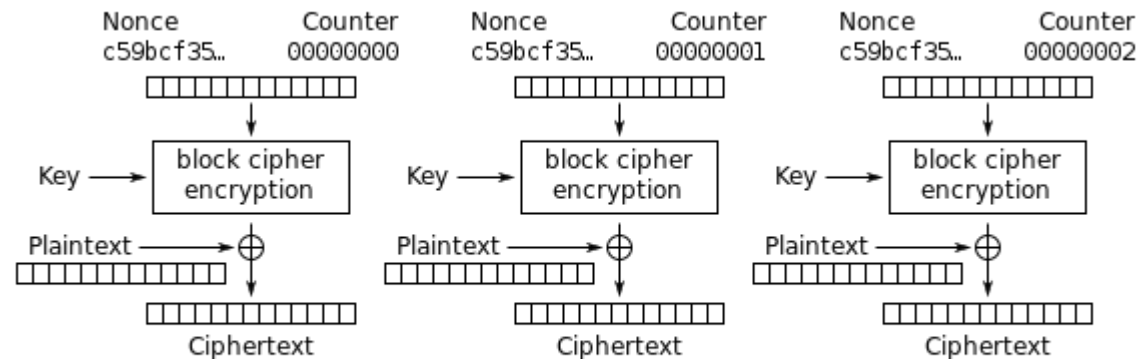
- An initialization vector (IV) is used to start the process (the IV need not be secret, but must not be reused)
 - Nonce, sequence number, date, ...
 - IV may be public, i.e., known to the attacker

Cipher Block Chaining Mode



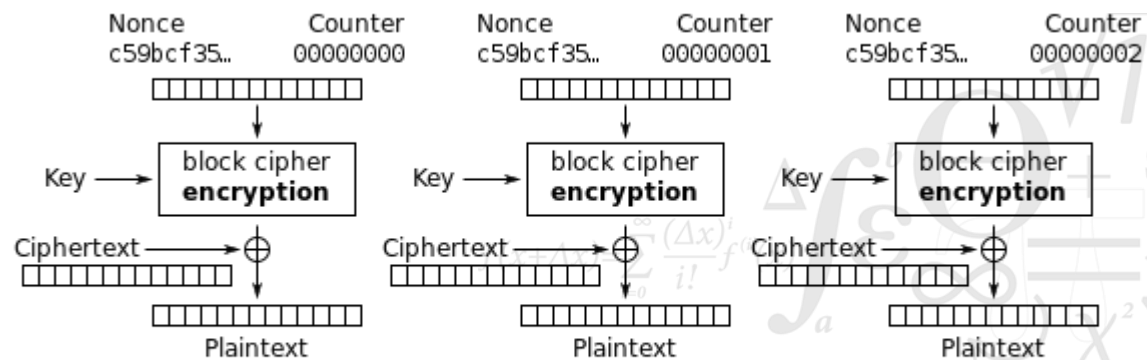
Counter Mode

- Encryption



Counter (CTR) mode encryption

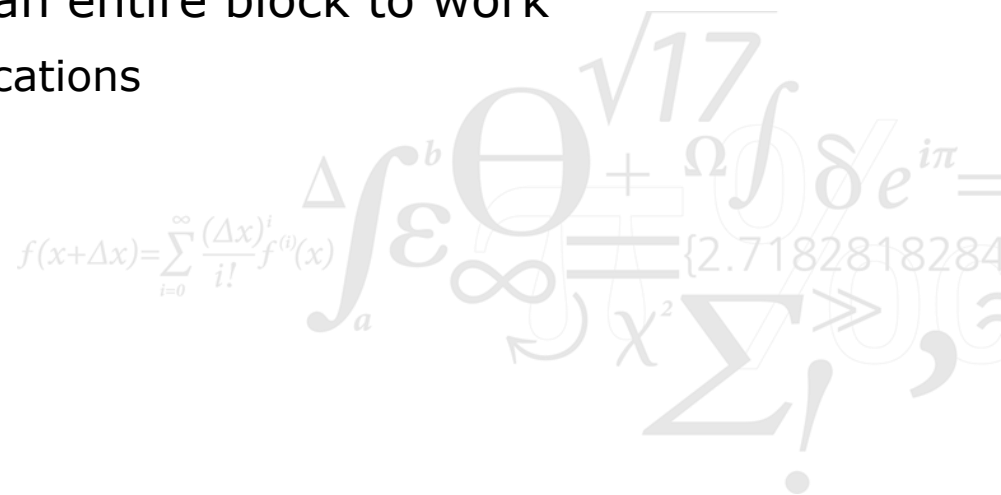
- Decryption



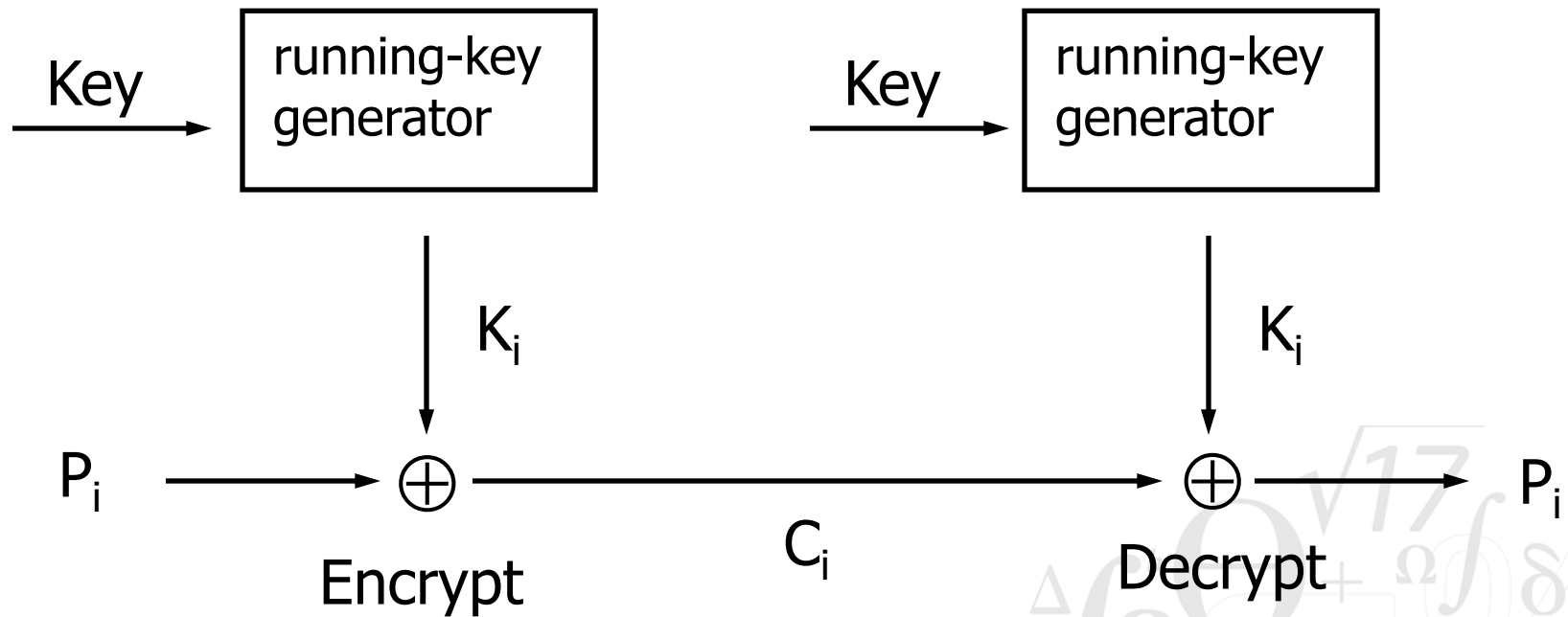
Counter (CTR) mode decryption

Stream Ciphers Algorithms

- Stream ciphers work on 1 bit/byte at the time
- A running-key generator generates a pseudo-random string of bits used for encryption
 - same running-key every time means that cryptanalysis is trivial
 - completely random running-key is equivalent to a one time pad (complete security, but not possible)
 - the key is used to seed the running-key generator
- Stream ciphers do not require an entire block to work
 - Good for character based applications

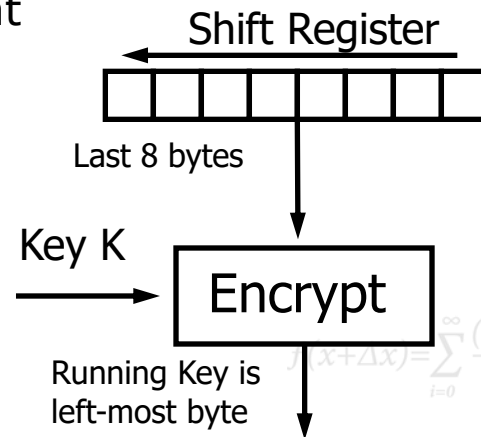


Stream Ciphers II



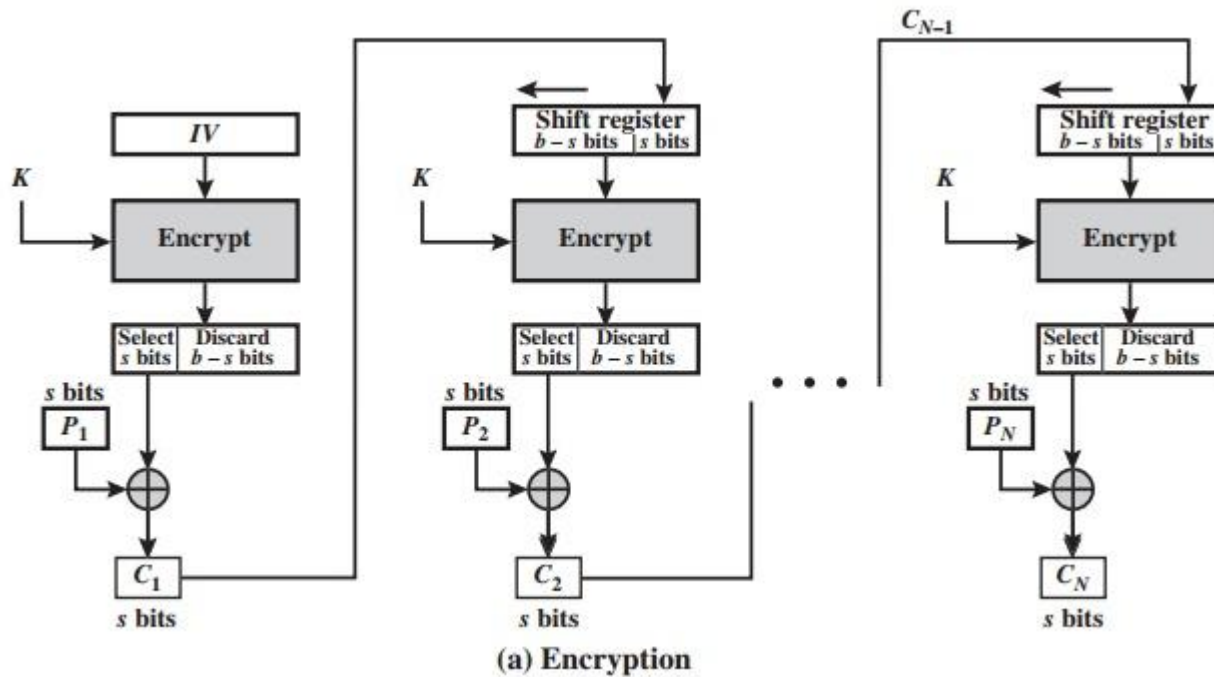
Creating Stream Ciphers from Block Ciphers

- Block ciphers can be used to implement stream ciphers
- Example: a 64bit block cipher is used to implement a byte stream cipher
 - A “shift register” (64bit) is encrypted and the leftmost byte is XORed with the plaintext byte
 - the shift register is shifted 1 byte to the left and the ciphertext byte is added to the right



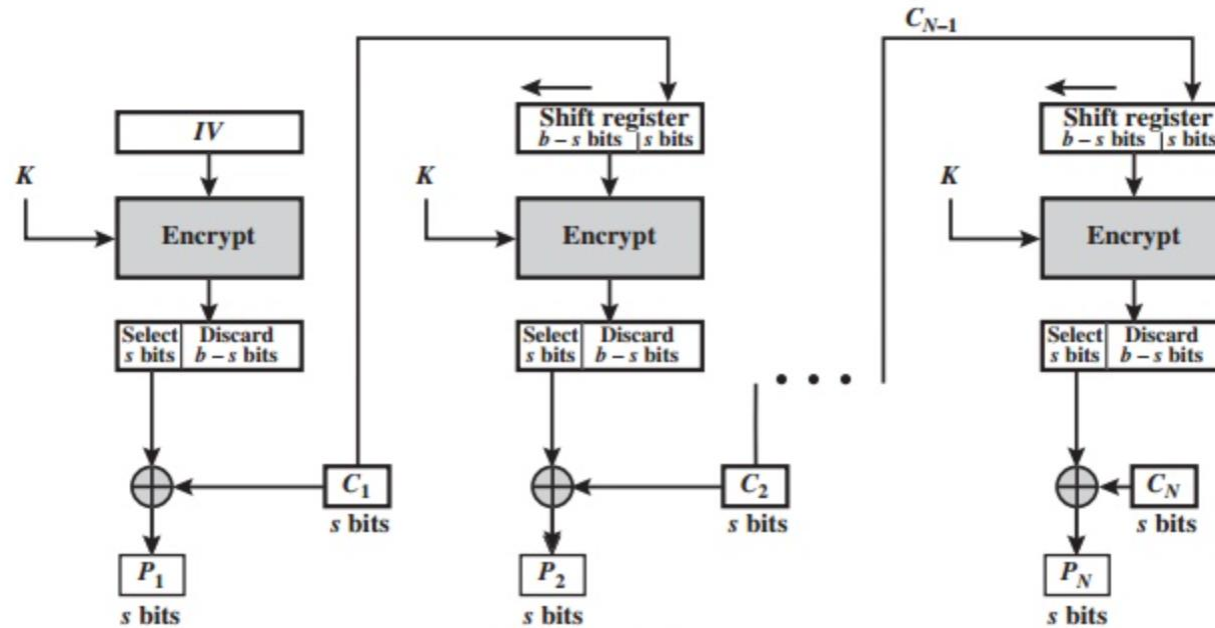
- The IV must be unique, e.g., a serial-number

Cipher Feedback Mode



CFB Encryption

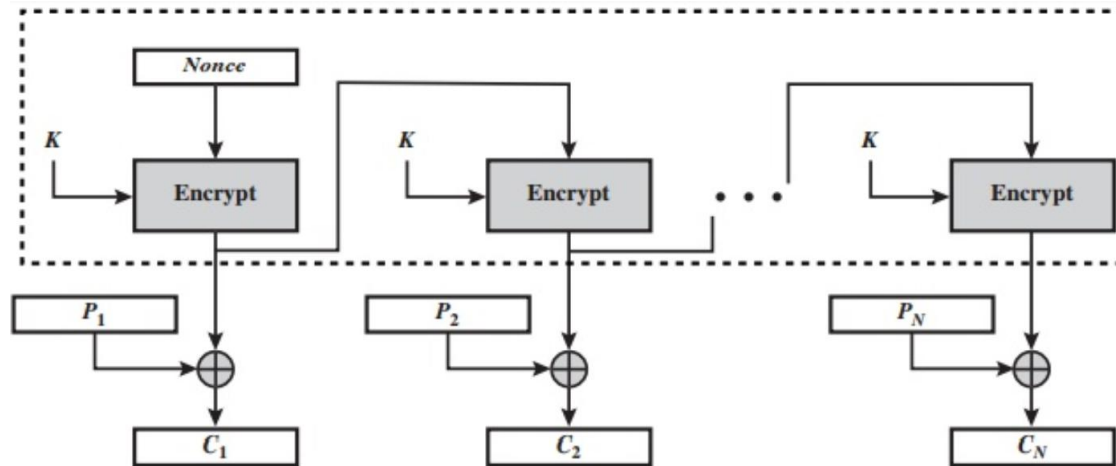
Cipher Feedback Mode



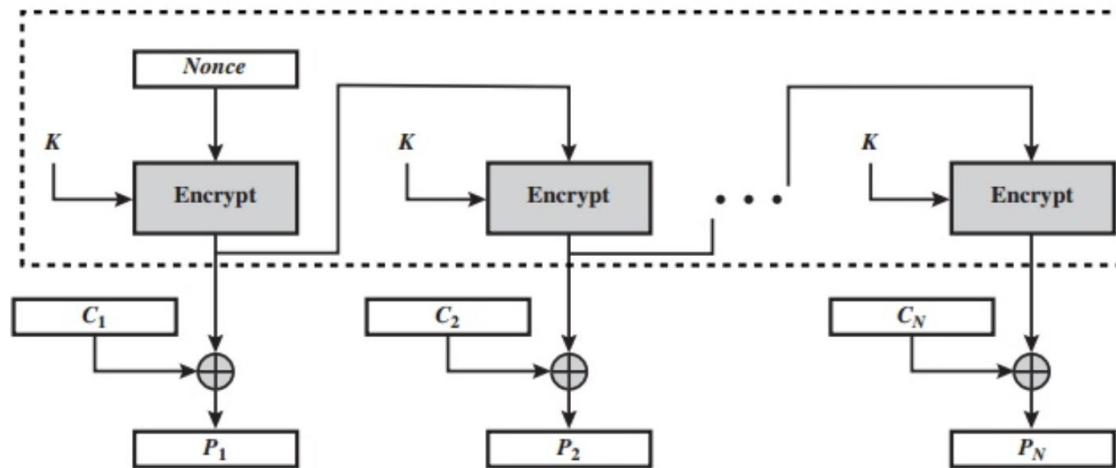
(b) Decryption

CFB Decryption

Output Feedback Mode



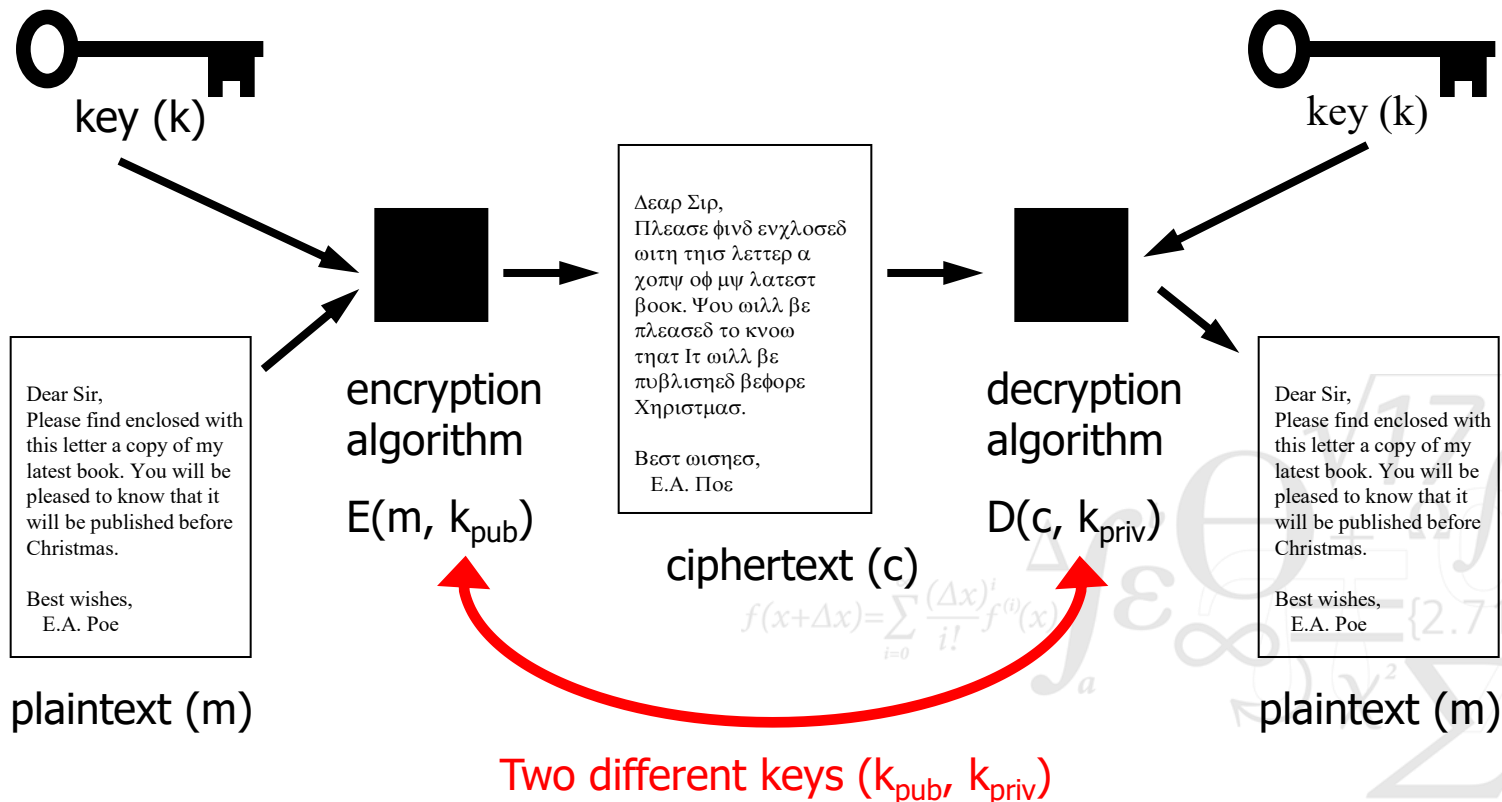
(a) Encryption



(b) Decryption

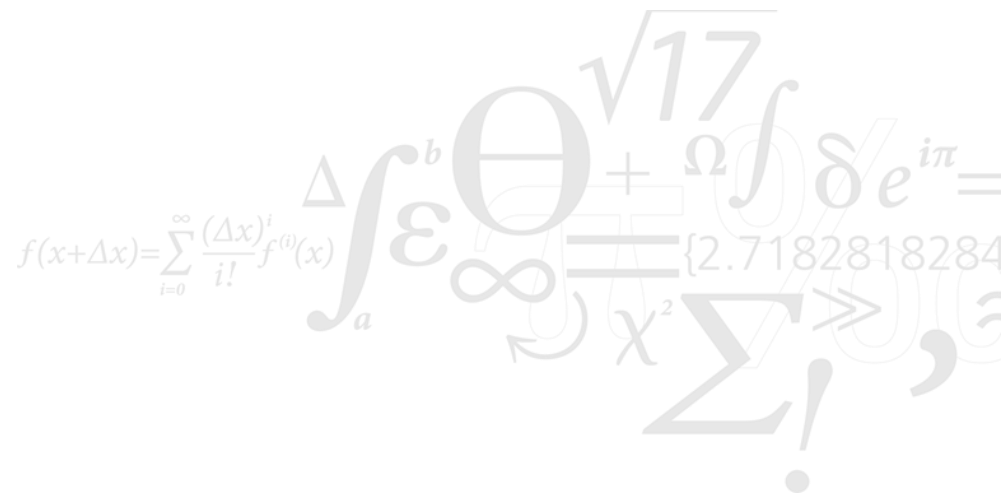
Asymmetric Cryptography

- Decryption-key cannot be derived from Encryption-key



Limitations

- Compared to symmetric cryptography, asymmetric cryptography is inefficient (slow, long keys)
 - In practice it is not used to encrypt large amounts of data
 - *Building block for key exchange*
 - *Building block for other protocols*



The receiver decrypts the modified ciphertext without detecting the manipulation.

- A. Encryption alone does not necessarily provide integrity/authentication
- B. AES cannot provide confidentiality
- C. Symmetric encryption cannot use keys
- D. CBC automatically detects all modifications

Sample Asymmetric Encryption Algorithms

- Asymmetric encryption is based on computationally hard problems
 - Problems that we cannot solve efficiently on computers
- Important groups of asymmetric encryption algorithms
 - RSA (prime factorization of large numbers)
 - Diffie-Hellman (modular arithmetic)
 - Diffie-Hellman (elliptic curves)
- Asymmetric encryption schemes define how algorithms are used in practice, i.e., arbitrary length messages, padding, etc.
- Particularly important: Semantic Security

Encryption scheme has to randomize the message, since otherwise, the following attacks become possible

 - Guess the message's content (guess M that corresponds to given C)
 - Use (known) public key to check whether the guess is correct

RSA

- Most popular public-key cryptosystem
- 1977: Rivest, Shamir, Adleman (RSA)
- Both encryption and authentication
- Survived years of attacks (cryptanalysis)
 - Probably secure
- Part of many official standards worldwide
 - Interoperability with existing code base



RSA overview

- Based on the difficulty of factorization
- Pick two random large primes: p and q $P=5, Q=11$
- Calculate $n = pq$ $n=55$
- Chose random encryption key e $e=3$
 - e and $(p-1)(q-1)$ must be relatively prime $3, 40$ -relatively Prime ?
- Compute decryption key d (extended Euclidian algorithm), such that

$ed \equiv 1 \pmod{(p-1)(q-1)}$
 $3d \equiv 1 \pmod{40}$
- Public key = (e, n) Private key = d

Private key $d=27$
 public key $(3, 55)$

Using RSA

- Encryption of a plaintext M
 - divide M into numerical blocks $m_i < n$
$$c_i = m_i^e \pmod{n}$$
- Decryption of ciphertext block c_i

$$m_i = c_i^d \pmod{n}$$

because

$$c_i^d = (m_i^e)^d = m_i^{ed} = m_i^1 \pmod{n}$$

Let's $m=7$

$$C = 7^3 \pmod{55}$$

$$C = 13$$

$$m = 13^{27} \pmod{55}$$

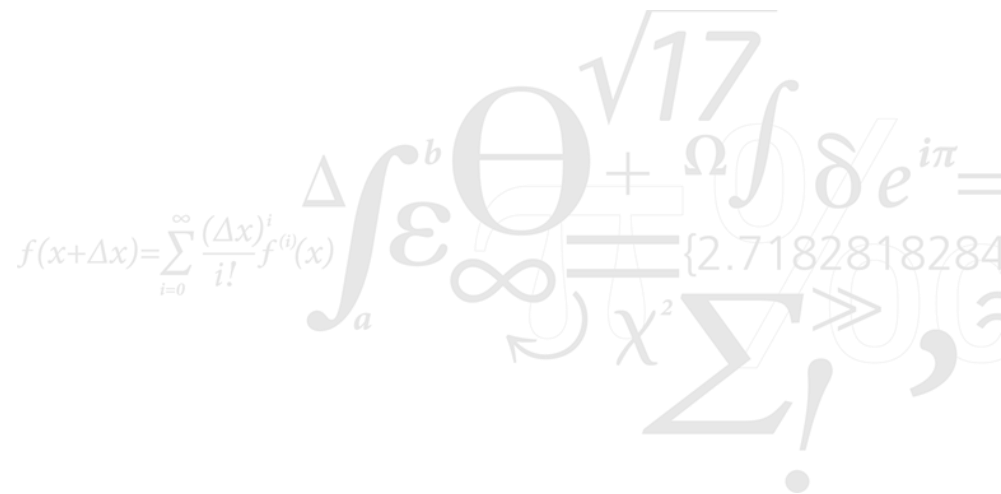
$$m = 7$$

ElGamal

- Based on the difficulty of calculating discrete logarithms in a finite field

$$y = g^x \bmod p$$

- p is prime
 - g and x are random numbers less than p
- Public key = (y, g, p)
- Private key = x



ElGamal Encryption

Encryption of message M

- Select random k relatively prime to $p - 1$

$$c1 = g^k \text{ mod } p$$

$$c2 = y^k M \text{ mod } p$$
- The pair (a,b) is the ciphertext

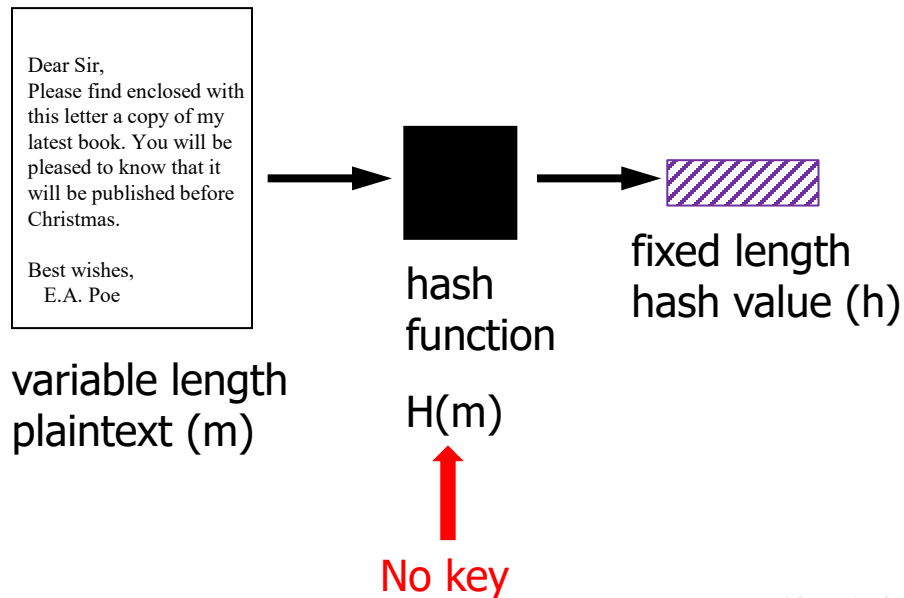
Decryption of a and b

$$s = c_1^x \text{ mod } p.$$

Compute modular inverse $s^{-1} \text{ mod } p$

$$M = c_2 \cdot s^{-1} \text{ mod } p.$$

Cryptographic Hash Functions



- Cryptographic hash functions must also satisfy:

1. Given M , computation of h is easy
2. Given h , it is intractable to compute M such that $H(M) = h$
3. It is intractable to find M and M' such that $H(M') = H(M)$

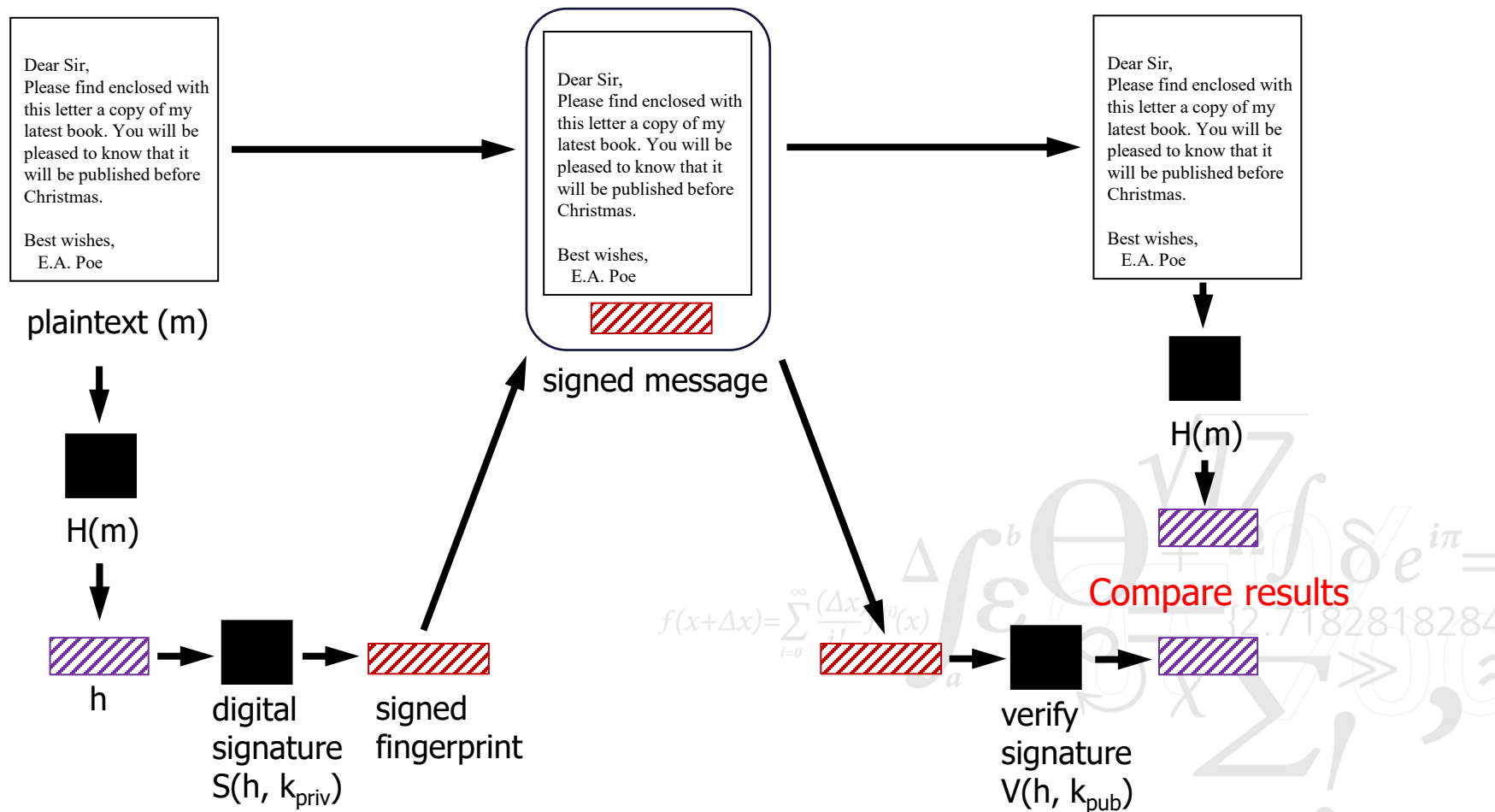
- Hash value h is often called a "fingerprint" or "digest" of M

Hash Functions

- The “work horses” of cryptography
- Main uses include:
 - Condensing long strings into short strings (collision resistant hash function)
 - Making an irreversible transformation without a key (one-way function)
- Prominent Examples:
 - MD4, MD5, SHA-0 (badly broken)
 - SHA-1 (broken, still found in standards, but must be retired before 2030)
 - SHA-2 (current standard)
 - SHA-3 (newest standard – since 2015)

Digital Signatures

- Operation is expensive, so we normally sign the fingerprint

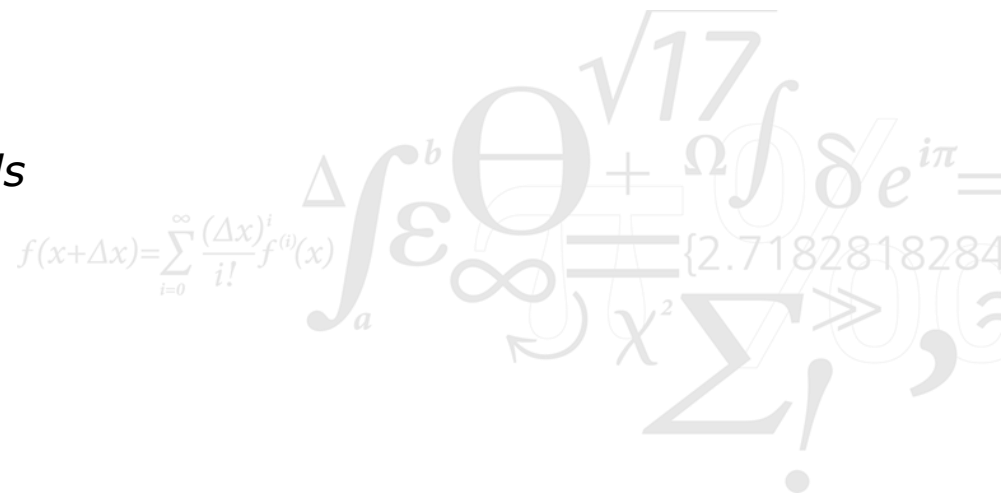


Security Level

- If the best known attack is equivalent to running the cryptographic algorithm 2^n times, then we have a security level of n bit
- Typical 80 bit (too low) 128 bit (decent), 256 bit (high)
- is security level always equal to the key length ????
- Important exceptions:
 - Hash functions (hash size $\geq 2 \times$ security level)
 - Asymmetric cryptography (e.g. RSA: 1024/2038/4096 bit)

Summary on Building Blocks

- Please remember the following
 - Clarify your security goals
 - Don't design your own cryptographic primitives and protocols
 - *Always rely on well known (and analysed) standards*
 - Make sure that you understand the primitives and protocols you use:
 - *Security goals*
 - *Security level*
 - *How to apply them*
 - *Known problems and pitfalls*





That's all Folks!